Project Cascade: Telecom-Routed AI Architecture for Godot 4

1. Core Operating Invariants

- **Stateless Compute Kernel:** The local LLM (Mule) maintains zero conversational history, memory buffers, or project context across inference boundaries.
- **Zero Natural Language Pipeline:** Raw natural language is completely barred from reaching the Mule. All instructions, constraints, and contextual facts are serialized into rigid JSON parameter blocks.
- **Separation of State and Inference:** Project topology, node state, and file locations reside strictly on disk inside deterministic JSON registries.
- **Strict Telecommunications Protocol:** Interaction follows a fixed Time-Division Multiple Access (TDMA) clock, subnet boundary firewalls, and Negative Acknowledgment (NAK) retry loops terminating in a Dead-Letter Queue (DLQ).

2. Network Topology & Addressing Map (`network_topology.json`)

All project nodes, resources, and RAG knowledge chunks are mapped into IPv4 address spaces to prevent namespace collisions and scope drift.

Subnet Allocation

- `10.0.0.0/16` (Core Backbone): Global Autoloads, singletons, master event buses, and global state machines.
- `10.1.0.0/16` (Domain 1 / Scene Root):
  - `10.1.1.0/24`: Dynamic actors (`10.1.1.10` = `%Player`, `10.1.1.11` = `%Weapon`).
  - `10.1.2.0/24`: Local NPCs, environmental interactables, and dynamic props.
- `10.2.0.0/16` (Domain 2): Completely isolated address space for secondary levels or external game states.
- `10.10.0.0/16` – `10.90.0.0/16` (RAG Knowledge Domains): Hard-partitioned data clusters (e.g., `10.10.x.x` = Mechanics API, `10.20.x.x` = Shader Library).
- `10.100.0.0/16` (Edge Sandbox): External plugins, mods, and experimental scripts.

Addressing & Routing Mechanics

- **MAC Address (`uid://`):** Godot 4 Resource UID. Permanent, content-addressed, and immune to filesystem moves or renames.
- **DNS Resolution (`%UniqueName`):** Godot Scene Unique Names. Resolved strictly by the Python harness through `state_registry.json` to absolute tree paths.
- **Pub/Sub Routing:** Typed Godot Signals emitted upward from child to parent nodes.
- **Multicast Distribution:** Godot Node Groups (`get_tree().call_group()`) for broad single-to-many event triggers.
- **Peering Exchange (IXP):** Cross-subnet direct pathing (`../../`) is physically blocked. Cross-domain data transfer must peer through `/root/InterconnectBus`.

3. The 4-Slot TDMA Master Clock Schedule

Every modification to the engine runs sequentially through four isolated phases managed by the Python harness (`olt_master_clock.py`).

[TDMA MASTER CLOCK FRAME]
├── SLOT 1: FETCH
│ ├── Action: Harness queries state_registry.json for the target address/UID.
│ └── Guard Interval 1: Verify UID exists; ensure 0 ARP/IP address collisions.
├── SLOT 2: ROUTE
│ ├── Action: Harness loads the target tool schema and pre-warms expert cache.
│ └── Guard Interval 2: Verify System RAM cache ≤ 6.0 GB and VRAM ≤ 10.0 GB.
├── SLOT 3: EXECUTE
│ ├── Action: Mule processes single task block + matched RAG record.
│ └── Guard Interval 3: Checksum scan (JSON parsing, exclude list, subnet isolation).
└── SLOT 4: COMMIT / NAK
  ├── PASS (ACK): Harness applies direct disk write; runs godot --headless verification.
  └── FAIL (NAK): Harness returns exact error packet (max 3 retries -> Dead-Letter Queue).

4. Formal Schema Definitions

Task Block Specification (`task_block_spec.json`)

The operational payload transmitted to the Mule during **Slot 3: Execute**:

JSON
```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "title": "CascadeTaskBlock",
 "type": "object",
 "required": [
```

```
    "frame_id",
    "time_slot",
    "subnet",
    "op",
    "target",
    "property",
    "type",
    "value",
    "ref",
    "exclude"
  ],
  "properties": {
    "frame_id": { "type": "string", "pattern": "^0x[0-9A-Fa-f]{4}$" },
    "time_slot": { "type": "string", "enum": ["SLOT_3_EXECUTE"] },
    "subnet": { "type": "string", "pattern": "^10\\.[0-9]{1,3}\\.[0-9]{1,3}\\.0/24$" },
    "op": {
      "type": "string",
      "enum": ["modify_property", "spawn_instance", "bind_signal", "insert_method"]
    },
    "target": { "type": "string", "pattern": "^%[A-Za-z0-9_]+$" },
    "property": { "type": "string" },
    "type": { "type": "string", "enum": ["Vector2", "Vector3", "float", "int", "bool", "String", "Signal"] },
    "value": {},
    "ref": { "type": "string", "pattern": "^10\\.[0-9]{1,3}\\.[0-9]{1,3}\\.[0-9]{1,3}$" },
    "exclude": {
      "type": "array",
      "items": { "type": "string" },
      "uniqueItems": true
    }
  },
  "additionalProperties": false
}
```

RAG Record Specification (`rag_record_spec.json`)

The structural context record retrieved during **Slot 1: Fetch** and injected alongside the task block:

JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "CascadeRAGRecord",
  "type": "object",
```

```
  "required": [
    "addr",
    "subnet",
    "uid",
    "node",
    "name",
    "props",
    "peering",
    "exclude"
  ],
  "properties": {
    "addr": { "type": "string", "pattern": "^10\\.[0-9]{1,3}\\.[0-9]{1,3}\\.[0-9]{1,3}$" },
    "subnet": { "type": "string", "pattern": "^10\\.[0-9]{1,3}\\.[0-9]{1,3}\\.0/24$" },
    "uid": { "type": "string", "pattern": "^uid://[a-z0-9]+$" },
    "node": { "type": "string" },
    "name": { "type": "string", "pattern": "^%[A-Za-z0-9_]+$" },
    "props": {
      "type": "object",
      "additionalProperties": { "type": "string" }
    },
    "peering": {
      "type": "object",
      "required": ["pub_sub", "multicast", "ixp"],
      "properties": {
        "pub_sub": { "type": "string" },
        "multicast": { "type": "string" },
        "ixp": { "type": "string", "const": "/root/InterconnectBus" }
      },
      "additionalProperties": false
    },
    "exclude": {
      "type": "array",
      "items": { "type": "string" },
      "uniqueItems": true
    }
  },
  "additionalProperties": false
}
```

Execution Telemetry & Error Packet (`nak_packet_spec.json`)

The payload returned to the harness if Guard Interval 3 or Slot 4 validation fails:

JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "CascadeNAKPacket",
  "type": "object",
  "required": ["frame_id", "status", "retry_count", "error_code", "violation_detail"],
  "properties": {
    "frame_id": { "type": "string" },
    "status": { "type": "string", "enum": ["NAK"] },
    "retry_count": { "type": "integer", "minimum": 1, "maximum": 3 },
    "error_code": {
      "type": "string",
      "enum": [
        "ERR_SYNTAX_INVALID",
        "ERR_EXCLUDE_LIST_VIOLATION",
        "ERR_CROSS_SUBNET_BOUNDARY",
        "ERR_COMPILATION_HEADLESS",
        "ERR_ARP_COLLISION"
      ]
    },
    "violation_detail": { "type": "string" }
  },
  "additionalProperties": false
}
```

5. Engine Modification Model

Changes are executed directly on disk by the Python harness rather than through an in-engine MCP server:

1. **Direct Serialization:** The harness reads and edits the `.tscn` (Text Scene) and `.gd` (GDScript) files directly on disk.
2. **Deterministic Judge:** After file modification, the harness issues an out-of-process subprocess command:
3. Bash

```
godot --headless --check-only --script <target_file.gd>
```

4.
5.
6. **Commit or Revert:** If the return code is `0`, an `ACK` is declared and `state_registry.json` is updated. If non-zero, the modification is reverted, the retry

counter increments, and a `NAK` frame is prepared. If attempts exceed 3, the payload is dumped to `failed_frames.jsonl` (Dead-Letter Queue).

Concrete Implementation Checklist

1. Create a clean project workspace directory:
2. DOS

```
mkdir C:\mule\project_cascade
cd C:\mule\project_cascade
```

3.
4.
5. Save the three JSON schema definitions above into `task_block_spec.json`, `rag_record_spec.json`, and `nak_packet_spec.json`.
6. Initialize the core network topology file (`network_topology.json`) defining the subnets for Godot root nodes and actor scenes.
7. Set up a minimal Godot 4 test project inside `C:\mule\project_cascade\engine_test` containing one `CharacterBody2D` with Scene Unique Name `%Player`.
8. Run an extraction script to read `%Player`'s `uid://` from the `.tscn` file and generate the first verified row of `state_registry.json`.